PACT

Technical architecture and implementation specification | Version 1.0 | 22 September 2026

Purpose. A build-ready blueprint for an autonomous-agent authority and economic accountability system on Monad. It defines the six protocol layers, contract state machines, policy engine, API gateway, SDKs, MCP server, Console, Explorer, proof formats, failure cases, delivery stages and acceptance tests.

Product thesis. Pact controls what agents can do before execution, proves their authority to counterparties, records the obligations they create and produces independently verifiable receipts for the eventual economic outcome.

Central invariant. An agent can request a privileged action but cannot execute it with an unrestricted principal key or upstream API credential. Pact must hold the execution boundary, or it cannot honestly claim to enforce policy.

Implementation status. This is an architecture and build specification, not a claim that contracts or integrations have been deployed. Network addresses, sponsor requirements and stablecoin availability must be checked immediately before implementation.

1. Product scope and boundaries

1.1 Actors

Actor	Role
Principal	Person, organization or smart account granting authority and funding actions.
Agent	Automated software with a distinct identity and restricted operational key.
Delegate	Child agent operating under narrower inherited authority.
Counterparty	Provider or buyer checking authority and signing obligations.
Verifier	Named party or mechanism judging delivery under specified rules.
Operator	Runs Pact services; cannot override contract capacity or forge principal signatures.

1.2 Guarantees and non-guarantees

• Hard enforcement covers assets in Pact-controlled accounts and API credentials held exclusively by the Pact vault. A separate unrestricted wallet or leaked raw API key can bypass Pact.

• An onchain hash proves integrity of disclosed content, not the truth of private evidence or legal enforceability of a service contract.

• Opaque contract calls and unknown APIs cannot receive a trustworthy exposure estimate. Default to denial until a reviewed adapter exists.

• Revocation stops new actions. Existing signed obligations remain governed by their cancellation, dispute and settlement terms.

• Approval signatures must bind exact terms and expiry; a general approval is never treated as blanket authorization.

2. System architecture

```
Principal -> Console -> Engine -> AuthorityRegistry / CommitmentLedger
Agent -> TS/Python SDK or MCP -> Engine -> [API Gateway | PactAccount]
Counterparty -> Authority proof verifier -> ObligationManager
Delivery -> Verifier -> SettlementVault -> ReceiptRegistry
Monad events -> Indexer -> Console / Explorer
```

Product	Responsibilities	Trust boundary
Pact Protocol	Onchain grants, delegation, capacity, wallet guard, obligations, escrow and receipt anchors.	Canonical economic state.
Pact Engine	Intent parsing, exposure adapters, simulation, policy, approvals and execution orchestration.	Can recommend; contracts independently enforce critical limits.
Pact SDK	Typed client libraries, signing, retries, reads and proof verification.	Never stores principal private keys.
Pact MCP Server	Scoped tools and resources for AI clients.	Prompt/tool invocation is not consent.
Pact Console	Human grants, policies, approvals, obligations and audit views.	Signing occurs on trusted principal surface.
Pact Explorer	Public proof and event verification with private disclosure controls.	Displays attestation type and provenance.

2.1 Data placement

Store	Data
Monad	Grant relationships, revocation epochs, ceilings, reservations, commitment amounts, obligation states, escrow and receipt hashes.
Encrypted database	Policy documents, approval requests, API metadata, evidence pointers and private organization settings.
Secrets vault	Upstream API keys, signing service credentials if any; envelope encryption, access audit and rotation.
Evidence storage	Encrypted evidence objects referenced by immutable content hashes; retention and access policy.
Indexer database	Rebuildable event projections; never authoritative for execution.

3. Six protocol layers

Layer	Question	Implementation
1 Identity and authority	Who may act for whom?	Agent identity binding, root grants, delegations, expiry and revocation.
2 Policy and authorization	Is this exact action permitted now?	Typed intents, access/finance/time/economic rules and approvals.
3 Proof of Authority	Can a third party verify capacity?	Short-lived action-bound proof backed by atomic reservation.
4 Proof of Obligation	What was agreed to?	Mutually signed terms plus authority reference and lifecycle state.
5 Commitment controls	What liability is consumed?	Exposure adapter, ancestor capacity checks, reservations and release.
6 Economic receipts	What happened in the end?	Linked authorization, execution, evidence, verdict and settlement records.

4. Canonical objects and signatures

4.1 Typed domain objects

```
Principal { address, kind, recoveryConfigHash }
AgentBinding { agentId, operationalKey, principal, registryRef, keyEpoch }
Grant { id, principal, agentId, keyEpoch, parentId, policyHash,
  validFrom, expiresAt, depthRemaining, revocationEpoch }
Intent { id, grantId, agentId, actionType, target, asset,
  payloadHash, termsHash, nonce, deadline, chainId }
ExposureQuote { adapterId, adapterVersion, cashNow, periodSpend,
  grossLiability, requiredReservation, pricingRef, validUntil }
Decision { intentHash, policyHash, result, reasons, quoteHash, expiry }
```

All quantities are integer base units and specify an asset or accounting unit. An exposure quote must state assumptions, oracle price freshness if used, cancellation penalties and a conservative upper bound. Policy evaluation is deterministic for a given policy version, intent, quote and chain snapshot.

4.2 Proof schemas

```
AuthorityProof { chainId, verifyingContract, principal, agentId,
  grantId, policyHash, intentHash, termsHash, asset,
  maximumLiability, reservationId, nonce, expiresAt }
ObligationProof { obligationId, authorityProofHash, termsHash,
  buyerSignature, providerSignature, state, createdBlock }
EconomicReceipt { receiptId, obligationId, grantId, intentHash,
  policyHash, approvalHash, executionTxs[], evidenceRoot,
  verdictHash, settlementTxs[], outcome, emittedBlock }
```

Use EIP-712 structured signatures with chain ID, verifying contract, nonce, typed payload and deadline. Verify contract-wallet principals through ERC-1271. For passkeys, validate P256 support and the chosen account implementation on the target Monad network before claiming native approval support. Do not expose private policy values unless deliberately disclosed; policy hashes alone do not make a proof meaningful without the verifier receiving the relevant disclosed constraints or trusting contract enforcement.

5. Pact Protocol on Monad

5.1 AuthorityRegistry

• createRootGrant: principal signature, unique nonce, current policy hash, positive limits and expiry.

• delegate: verify child capabilities, duration, budget, depth, target sets and risk class are subsets of the parent; allocate child capacity in the ledger.

• revoke: increment epoch or mark grant revoked; descendants become invalid for new execution through ancestor validation.

• rotateAgentKey: principal-controlled binding with key epoch; old signatures stop working for new intents.

• isActive: checks grant, binding, time and every ancestor. Bound delegation depth to keep onchain verification predictable.

5.2 CommitmentLedger and accounting invariants

```
available(g) = ceiling(g) - childAllocations(g)
             - pendingReservations(g) - activeCommitments(g)
Invariant: all values >= 0 and available(g) >= 0
Invariant: a child action counts against its allocation AND relevant ancestors
Invariant: a reservation can be consumed once or expire once
Invariant: release cannot exceed outstanding reserved/committed amount
```

This formula represents one liability ceiling in one accounting unit. Independent counters track immediate spend, rolling-period settled spend, and optional adapter-defined exposure. Escrowed value is tracked separately from gross contractual liability. Settlement reduces remaining liability only under explicit terms; recurring capacity does not regenerate merely because a partial payment cleared.

Example: Agent A has 15,000 USDC liability ceiling; 6,000 is allocated to Agent B. A has 9,000 unallocated before other commitments; B can reserve at most 6,000. A child action consumes capacity in B and its ancestors. A proposed 6,000 agreement under a 500 child allocation must fail or request a new principal-authorized allocation.

5.3 PactAccount: controlled execution

• Accept only recognized action adapters in version 1: token transfer, approved contract call and escrow funding.

• Revalidate grant, ancestor status, intent nonce, exact target, selector, amount, asset, policy hash, expiry, reservation and approval in the same transaction.

• Reject arbitrary delegatecall, blanket ERC-20 approval, unreviewed multicall and opaque router operations.

• Use non-reentrancy controls and checks-effects-interactions; never rely only on an earlier offchain simulation.

• Treat a failed execution as failed: do not finalize commitment or receipt as successful. Provide explicit reservation expiry and recovery.

5.4 ObligationManager and SettlementVault

```
PROPOSED -> RESERVED -> ACTIVE -> SUBMITTED -> VERIFIED -> SETTLED
                    |         |            |          |
                 EXPIRED   CANCELLED     DISPUTED -> RESOLVED -> SETTLED
```

Agreement terms include parties, service specification hash, milestones, asset, price, gross maximum liability, deposit schedule, due dates, evidence types, verifier, challenge period, cancellation/termination rules, refund routing and dispute resolver. Both parties accept the identical terms hash. Funds may be

reserved as authority without being fully escrowed; display the exact funded amount and remaining credit exposure.

5.5 ReceiptRegistry

Anchor a versioned event graph: grant -> intent -> decision -> approval -> obligation -> execution -> evidence -> verdict -> settlement. Store hashes and references, not raw secrets or personally identifying evidence. A receipt states whether a result is contract-proven, externally attested, challenged or unresolved. A public verifier reconstructs the graph from contract events and supplied disclosure material.

5.6 Indicative Solidity interfaces

```
interface IAuthorityRegistry {
  function createGrant(GrantInput calldata, bytes calldata signature) external returns (uint256);
  function delegate(uint256 parentId, GrantInput calldata) external returns (uint256);
  function revoke(uint256 grantId) external;
  function isActive(uint256 grantId, address agentKey) external view returns (bool);
}
interface ICommitmentLedger {
  function reserve(uint256 grantId, bytes32 intentHash, uint256 amount,
                   uint64 expiry) external returns (uint256);
  function commit(uint256 reservationId, uint256 obligationId) external;
  function release(uint256 obligationId, uint256 amount) external;
  function available(uint256 grantId) external view returns (uint256);
}
interface IObligationManager {
  function create(bytes32 termsHash, uint256 reservationId,
                  bytes calldata partySignatures) external returns (uint256);
  function submit(uint256 id, bytes32 evidenceRoot) external;
  function resolve(uint256 id, bytes32 verdictHash) external;
}
```

Interfaces are design guidance. Implementation must define access modifiers, exact structs, supported assets, role separation and upgradeability before code is treated as production ready. Prefer minimally upgradeable custody contracts or explicit timelocked upgrades with principal opt-out.

6. Pact Engine and policy semantics

6.1 Engine pipeline

```
Authenticate agent -> normalize intent -> select reviewed adapter
-> fetch current grant/policy -> decode action -> simulate if onchain
-> calculate worst-case exposure -> evaluate policy -> reserve/approve
-> revalidate onchain -> execute -> reconcile -> issue receipt
```

Policy class	Sample controls
Access	Chain, target contract, function selector, API host, exact endpoint and HTTP method.
Financial	Single transaction, per-period spend, asset/recipient, gas ceiling and approval threshold.
Temporal	Grant validity, business hours, cooldown, rate limits and expiry.
Delegation	Child ceiling, allowed capabilities, depth, target set and cascading revocation.
Economic	Maximum future liability, term, leverage restriction, recurrence, cancellation penalties.

6.2 Exposure adapters

• Transfer adapter: cashNow equals token amount plus bounded fees; maximum liability normally equals the authorized transfer.

• Recurring agreement adapter: sum maximum payable installments plus enforceable termination penalties, avoiding double count where penalty replaces remaining installments.

• Escrow agreement adapter: gross liability and locked collateral shown separately; compute actual unsettled liability after each milestone.

• Leverage adapter: only support specific reviewed venues whose position exposure can be bounded; otherwise deny.

• Unknown action: UNSUPPORTED_ACTION, not ALLOW. Simulation reports execution effects at a snapshot; it does not prove economic safety.

6.3 Policy decision order

Deny on inactive grant, invalid ancestry, unsupported adapter, insufficient capacity, forbidden target, forbidden asset, excessive duration, prohibited risk class or expired policy. Then require approval when a valid action crosses a configured threshold. Allow only when every relevant condition passes. Any engine outage or stale quote fails closed for privileged execution.

6.4 Approval lifecycle

PENDING -> APPROVED or REJECTED or EXPIRED -> CONSUMED. Bind approval to intent hash, quote hash, terms hash, grant ID, maximum spend and liability, principal, chain, nonce and expiry. A material renegotiation invalidates prior approval. Approval never enlarges a grant ceiling unless the principal separately signs a new grant.

7. External API and credential enforcement

```
Agent -> Pact access token -> Gateway -> policy and payload checks
      -> budget reservation -> vaulted upstream credential -> provider
      -> provider result -> reconciliation -> receipt
```

Gateway security: exact host allowlist; DNS rebinding and redirect checks; reject unapproved redirects; parse and validate supported payload fields; attach idempotency keys; rate limit by agent and principal; redact secret-bearing headers; log request and response digests; rotate provider credentials; never return raw credentials to agents. Network egress from the agent runtime should be restricted where possible, but the fundamental requirement is that agents never receive upstream credentials.

For nondeterministic billing, reserve an enforceable maximum before forwarding; reconcile actual cost with provider usage records and release the difference. If a provider cannot give a bounded maximum, policy must require a conservative cap or deny the request. External providers do not automatically honor Pact obligations; their integration must specify whether the relationship is enforceable by contract, escrow or only logged as an attestation.

8. TypeScript and Python SDK

8.1 API contract

```
quote(intent) -> ExposureQuote
authorize(intent) -> DENY | REQUIRE_APPROVAL | ALLOW
requestApproval(intent, quote) -> approvalId
reserveAuthority(intent, termsHash) -> AuthorityProof
verifyAuthority(proof) -> verificationResult
createObligation(proof, signedTerms, funding) -> obligationId
submitEvidence(id, evidence) -> evidenceHash
challenge(id, reason) -> disputeId
settle(id) -> transactionReceipt
getEconomicReceipt(id) -> receipt
delegate(parentId, childGrant) / revoke(grantId)
```

Shared JSON Schema or protobuf definitions are generated into TypeScript and Python. Include schema version, backward-compatible extension fields, canonical hash encoding, UTC times, integer asset units, chain ID and typed error codes. Signature verification and public proof verification should work without Pact-hosted API access.

8.2 Error classes and retries

Use POLICY_DENIED, APPROVAL_REQUIRED, UNSUPPORTED_ACTION, INSUFFICIENT_CAPACITY, CAPACITY_CHANGED, QUOTE_EXPIRED, RESERVATION_EXPIRED, REVOKED, NONCE_USED, UPSTREAM_FAILED, EXECUTION_REVERTED and DISPUTED. Read retries may be automatic; mutation retries require an idempotency key and current-chain reconciliation. Never return an ALLOW response as a claim that execution or settlement succeeded.

9. MCP Server

Expose read tools for authority, quote, obligations and receipt lookup; mutation tools for proposal, approval request, approved execution, evidence submission and settlement initiation. Each tool checks a scoped Pact identity; tools do not inherit principal authority from a Claude/Codex prompt. The server must enforce the same policy path as SDK calls, with separate transport authentication, user confirmation when required and minimal tool output.

```
pact_get_my_authority | pact_quote_action | pact_propose_obligation
pact_request_approval | pact_execute_approved_action
pact_submit_evidence | pact_get_receipt
```

Prompt injection in evidence, retrieved pages or counterparties must not change the policy engine or tool authorization. Bind MCP sessions to agent IDs and principals; audit each tool invocation; avoid exposing unrelated grants or secrets. Read-only discovery can be broad only where disclosure policy explicitly allows it.

10. Pact Console and Explorer

10.1 Console screens

Screen	Required function
Overview	Available, delegated, reserved and committed capacity; active obligations; pending approvals.
Agents	Identity, operational keys, status, parent delegation and key rotation.
Policies	Human-readable editor, typed constraints, version diff, simulation and signed publishing.
Approvals	Exact intent, counterparty, total liability, cash now, duration, deadline and sign/deny.
Obligations	Terms, evidence, verifier, funding, milestones, challenge and settlement.
Audit	Searchable timeline with Monad transactions and offchain attestation origins.

10.2 Explorer verification

Provide proof pages for grant, reservation, obligation and receipt. Verify directly from Monad events and public contract reads. Show policy hash, disclosure status, verifier type, challenge status, funding versus gross liability and exact transaction links. Private terms remain encrypted; a viewer receives them only after authorization or counterparty disclosure. Never describe an undisclosed evidence hash as a successful independent verification.

11. End-to-end workflow

11.1 Demo scenario

Principal grants ProcurementAgent A a 15,000 USDC future-liability ceiling, a 2,000 USDC single immediate-payment limit, six-month maximum term and approval for commitments above 5,000 USDC. A initially delegates only 500 USDC to Agent B. B proposes 1,000 USDC/month for six months. The 6,000 USDC gross commitment fails at B's delegation boundary even though A has enough capacity. The principal creates a new 6,000 USDC child allocation; the quote now requires approval. The signed exact-terms approval produces a reservation. Provider verifies authority, accepts terms, delivers milestones, a specified verifier evaluates evidence, the vault settles and the linked receipt becomes inspectable.

11.2 Transaction sequence

```
1 principal signs grant and funds controlled account
2 agent submits intent and terms; engine quotes exposure
3 engine returns DENY, ALLOW or REQUIRE_APPROVAL
4 principal signs exact approval when necessary
5 ledger atomically reserves authority and emits reservation event
6 provider verifies reservation-backed AuthorityProof
7 parties sign terms; ObligationManager consumes reservation
8 provider submits evidence; verifier resolves or dispute opens
9 SettlementVault releases/refunds according to final state
10 ReceiptRegistry anchors linked outcome; indexer builds read view
```

12. Threat model and failure handling

Threat / failure	Required control
Agent bypasses policy	No unrestricted account key or raw API secret; restrict alternative egress where feasible.
Concurrent oversubscription	Atomic ledger updates and ancestor checks; reservation nonce and expiry.
Approval replay / changed terms	Typed signatures binding hash, nonce, chain, contract and expiry.
Unknown onchain economic effect	Reviewed adapters; arbitrary calls denied by default.
Prompt injection / tool abuse	Policy engine outside LLM; scoped MCP tools; never trust retrieved content as authority.
Provider outage after reservation	Idempotency and expiry; reconcile provider state before releasing budget.
False delivery attestation	Named verifier, signed evidence, challenge period, clear attestation label.
Revocation during agreement	Block new commitments while preserving explicit termination/settlement rights.
Index lag or RPC outage	Contracts are canonical; show pending state and never execute from stale index.
Upgrade or operator compromise	Minimize upgrade rights; timelock custody changes; audit logs and emergency pause scope.

12.1 Safety invariants for audits

- No execution without a currently valid grant and action-bound authorization.
- No child effective authority exceeds any ancestor, including after policy changes.
- No reservation or commitment oversubscribes any applicable ancestor ceiling.
- An approval or reservation is consumed at most once.
- No escrow withdrawal exceeds deposited and unsettled balance.
- A settled obligation cannot be paid or refunded twice.
- An offchain Gateway success is not represented as an onchain settlement.
- Revocation cannot retroactively erase or silently pay an obligation.

13. Implementation plan and acceptance gates

Stage	Deliverables	Exit evidence
0 Foundation	Monorepo, shared schemas, Monad local/testnet configuration, CI and action hashing.	Deterministic fixtures and contract deployments.
1 Authority	Registry, attenuation, ancestor revocation, ledger reservations, PactAccount.	Adversarial tests for bypass, replay and concurrent capacity.
2 Obligations	Terms signing, escrow, milestone states, challenge flow and receipts.	End-to-end two-party funded agreement.
3 Engine	Adapters, simulation, policy compiler, approval service and reconciliation.	Denied unsafe call and approved bounded commitment.
4 Developer tools	TS/Python SDK, MCP server and API Gateway with one provider.	Sample agent cannot access raw secrets or bypass wallet policy.
5 Product	Console, Explorer and independently verifiable proof page.	Live demo and public proof verification from Monad.
6 Hardening	Security review, fuzzing, runbooks, observability and deployment docs.	No unresolved critical findings; recoverable failure flows.

13.1 Recommended repository layout

```
apps/console  apps/explorer  apps/api  apps/gateway  apps/mcp
packages/contracts  packages/engine  packages/schemas
packages/sdk-ts  packages/sdk-python  packages/proof-verifier
services/indexer  services/worker  examples/procurement-agent
tests/invariants  tests/e2e  docs/architecture  docs/protocol
```

13.2 Minimum demo acceptance tests

- A delegated 500 limit rejects a 6,000 liability even with a 1,000 first installment.
- Two competing agents cannot reserve the same remaining capacity.
- A changed term or revoked parent invalidates a pending proof before execution.
- The wallet rejects an unsupported contract call and an unapproved token transfer.
- The API Gateway rejects an unapproved method and never reveals the upstream secret.
- Counterparty verifies a reserved action without trusting the Console.
- A dispute holds settlement and the final receipt reflects the actual outcome.

14. Standards and integration decisions

Reference	Pact treatment
ERC-8004	Use existing agent identity and optional reputation/validation references; do not reinvent registry semantics.
AP2	Map commerce mandates/receipts when useful; Pact adds delegation-linked liability and obligation state.
ERC-7715	Permission request interoperability where wallet implementation supports it; not the liability ledger.
EIP-712 / ERC-1271	Typed signatures and contract-wallet verification; include independent replay protection.
MCP	Transport and tool protocol; each write action still goes through Pact authorization.
Sponsor bounties	Optional integrations only after reading each sponsor's actual bounty criteria; no eligibility assumed.

14.1 Sources checked for this specification

Monad documentation: https://docs.monad.xyz/ and https://docs.monad.xyz/developer-essentials/changelog ; ERC-8004: https://eips.ethereum.org/EIPS/eip-8004 ; ERC-7715: https://eips.ethereum.org/EIPS/eip-7715 ; EIP-712: https://eips.ethereum.org/EIPS/eip-712 ; AP2: https://ap2-protocol.org/ap2/agent_authorization/ ; MCP authorization: https://modelcontextprotocol.io/docs/tutorials/security/authorization ; Metropolis: https://monad.xyz/metropolis . Confirm deployment-network specifics and current sponsor terms before implementation.

15. Instructions to the implementing Codex agent

Implement a complete vertical slice before expanding features: grant -> bounded agent intent -> exposure quote -> approval -> atomic reservation -> counterparty proof verification -> signed obligation -> funded milestone -> verification -> settlement -> independently checked receipt. Preserve the safety invariants above. Begin by creating exact schemas and executable tests for accounting and revocation. Report every unsupported feature clearly. Do not substitute policy JSON, a dashboard mock or an offchain database for the account execution boundary and onchain commitment ledger. Request explicit product choices only when an assumption changes custody, verifier trust, asset support or dispute rights; otherwise use the conservative defaults in this specification.